

Applied Mathematics for ML

Quick Recap — Simple + Expert Definitions, with Types

Module 1: Linear Algebra Fundamentals

a. Scalars

SIMPLE:

A scalar is just one single number, like 5 or 0.01.

EXPERT:

A scalar is a zero-dimensional quantity representing pure magnitude with no directional component, used in ML for values like learning rate, loss, and bias terms.

b. Vectors

SIMPLE:

A vector is a list of numbers, like [3, 5, 2], that can represent a point or direction.

EXPERT:

A vector is a one-dimensional ordered array of numbers representing magnitude and direction in n-dimensional space, commonly used in ML to represent feature sets or embeddings.

TYPES:

- **Row vector** — numbers arranged horizontally
- **Column vector** — numbers arranged vertically

c. Matrices

SIMPLE:

A matrix is a grid of numbers with rows and columns, like a table.

EXPERT:

A matrix is a two-dimensional array of numbers organized into rows and columns, used in ML to represent entire datasets, images, or the weights of a neural network layer.

TYPES:

- **Square matrix** — equal rows and columns
- **Identity matrix** — 1s on the diagonal, 0s elsewhere, acts like multiplying by 1
- **Zero matrix** — every entry is 0
- **Diagonal matrix** — non-zero values only on the diagonal
- **Symmetric matrix** — equals its own transpose
- **Sparse matrix** — mostly zero values
- **Dense matrix** — mostly non-zero values

d. Matrix Operations

SIMPLE:

These are the basic things you can do to a matrix, like adding, scaling, or flipping it.

EXPERT:

Matrix operations include addition, subtraction, scalar multiplication, transposition, determinant calculation, and inversion, forming the foundational toolkit for manipulating structured data in ML pipelines.

TYPES:

- **Addition/Subtraction** — combine two same-shaped matrices element by element

- **Scalar multiplication** — multiply every element by one number
- **Transpose** — flip rows into columns
- **Inverse** — a matrix that undoes another matrix (only for some square matrices)
- **Determinant** — a single number summarizing key properties of a square matrix

e. Matrix Multiplication

SIMPLE:

Matrix multiplication combines two matrices by multiplying rows by columns and adding the results.

EXPERT:

Matrix multiplication is a non-commutative operation combining an $m \times n$ matrix with an $n \times p$ matrix via row-column dot products, forming the computational core of every neural network layer's forward pass.

TYPES:

- **Matrix x Matrix** — standard combination, requires $\text{cols}(A) = \text{rows}(B)$
- **Matrix x Vector** — special case producing one output vector, core of neural net layers
- **Elementwise (Hadamard)** — multiplies matching positions only, different from true matrix multiplication

Module 2: Vector Mathematics

a. Vector Representation

SIMPLE:

A vector can be written as a list of numbers or drawn as an arrow from the origin to a point.

EXPERT:

Vector representation refers to expressing a quantity as an ordered tuple of components, geometrically interpretable as a directed line segment from the origin, forming the basis of feature encoding in ML.

b. Magnitude

SIMPLE:

Magnitude is how long a vector is, found using the Pythagorean theorem.

EXPERT:

Magnitude (or norm) is the scalar length of a vector, computed as the square root of the sum of squared components, and is used in ML for normalization so that vector scale does not bias model computations.

c. Direction

SIMPLE:

Direction is which way a vector points, like a compass angle.

EXPERT:

Direction describes the orientation of a vector in space, independent of its magnitude, and is the property isolated by cosine similarity when comparing embeddings regardless of their length.

d. Dot Product

SIMPLE:

The dot product multiplies matching numbers from two vectors and adds them into one number.

EXPERT:

The dot product is a scalar-valued operation defined as the sum of elementwise products of two equal-length vectors, forming the mathematical basis of every neuron's weighted input computation.

e. Similarity Concepts

SIMPLE:

Similarity tells you how alike two vectors are, most commonly using cosine similarity.

EXPERT:

Similarity concepts quantify the closeness or likeness of two vectors using measures such as cosine similarity, forming the mathematical foundation of recommendation systems, semantic search, and clustering.

TYPES:

- **Cosine similarity** — angle-based, ignores magnitude; higher = more alike
- **Euclidean distance** — straight-line distance; lower = more alike
- **Manhattan distance** — grid-based distance; lower = more alike
- **Jaccard similarity** — overlap between two sets; higher = more alike
- **Pearson correlation** — how linearly two variables move together
- **Hamming distance** — count of differing positions in equal-length vectors

Module 3: Matrix Applications in ML

a. Data Representation

SIMPLE:

A dataset is stored as a matrix, where each row is one example and each column is one feature.

EXPERT:

Data representation in ML formalizes a dataset as a design matrix X of shape $(n_samples, n_features)$, the standard structural input expected by nearly every machine learning algorithm.

TYPES:

- **Dense representation** — every value stored, used for most tabular data
- **Sparse representation** — only non-zero values stored, used for text/bag-of-words data

b. Feature Matrices

SIMPLE:

A feature matrix holds only the input columns (X), separate from the answer column (y).

EXPERT:

A feature matrix is the input data matrix X used for model training, kept distinct from the label vector y , with categorical variables typically transformed via one-hot encoding before inclusion.

c. Transformations

SIMPLE:

A transformation changes vectors using a matrix, such as scaling, rotating, or normalizing data.

EXPERT:

A transformation is a matrix-based operation that maps vectors from one space to another, commonly used in ML for feature scaling, dimensionality change, or geometric manipulation of image data.

TYPES:

- **Scaling** — stretching or shrinking values
- **Rotation** — turning a vector around a fixed point
- **Translation** — shifting a vector's position
- **Reflection** — flipping a vector across an axis
- **Normalization** — rescaling values into a common range, usually 0 to 1

d. Matrix-Based Computations

SIMPLE:

Instead of looping through data one row at a time, one big matrix multiplication handles all rows at once.

EXPERT:

Matrix-based computation replaces iterative row-wise operations with vectorized matrix multiplication ($X @ weights$), enabling massive parallelization on GPU hardware and forming the computational backbone of deep learning.

e. Real-World ML Applications

SIMPLE:

Matrices power real systems like image recognition, recommendations, and spam detection.

EXPERT:

Real-world ML applications of matrix operations span convolutional image processing, collaborative-filtering recommendation engines, transformer attention mechanisms, and linear classifiers such as spam detectors.

Module 4: Calculus Intuition

a. Functions and Rates of Change

SIMPLE:

A rate of change tells you how much the output changes when the input changes a little.

EXPERT:

A rate of change quantifies the sensitivity of a function's output to variation in its input, distinguished into average (interval-based) and instantaneous (point-based) forms.

TYPES:

- **Average rate of change** — change measured over a whole interval
- **Instantaneous rate of change** — change measured at one exact point (a derivative)

b. Slopes

SIMPLE:

Slope measures steepness — how much y changes for every 1 unit x changes.

EXPERT:

Slope is the ratio of vertical change to horizontal change between two points on a function, and a slope of zero identifies a flat point corresponding to a local minimum, maximum, or saddle point.

TYPES:

- **Positive slope** — line rises as x increases
- **Negative slope** — line falls as x increases
- **Zero slope** — flat point, marks a minimum, maximum, or saddle
- **Undefined slope** — a vertical line

c. Derivatives (Conceptual)

SIMPLE:

A derivative is the slope at one exact point on a curve.

EXPERT:

A derivative is the instantaneous rate of change of a function at a given point, formally the limit of the average rate of change as the interval shrinks to zero, and underlies gradient computation in backpropagation.

TYPES:

- **First derivative** — the rate of change itself (e.g., speed)
- **Second derivative** — the rate of change of the rate of change (e.g., acceleration)
- **Partial derivative** — rate of change with respect to one input, holding others fixed

d. Optimization Intuition

SIMPLE:

Optimization means finding the input that makes a function's output as small or as large as possible.

EXPERT:

Optimization intuition frames model training as a search for critical points where the derivative equals zero, distinguishing local optima from the global optimum across a potentially non-convex loss surface.

TYPES:

- **Local minimum** — lowest point in a nearby region

- **Global minimum** — lowest point across the entire function
- **Local maximum** — highest point in a nearby region
- **Saddle point** — flat in one direction but not a true min or max

e. Why Calculus Matters in ML

SIMPLE:

Calculus tells a model which direction and how much to adjust its weights to reduce error.

EXPERT:

Calculus provides the derivative-based mechanism, via the chain rule in backpropagation, by which models systematically adjust parameters to minimize a loss function rather than relying on random search.

Module 5: Optimization Fundamentals

a. Cost Functions

SIMPLE:

A cost function measures how wrong a model's predictions are; lower is better.

EXPERT:

A cost function (loss function) is a scalar-valued measure of aggregate prediction error across a dataset, whose minimization defines the objective of the model training process.

TYPES:

- **MSE (Mean Squared Error)** — squares errors, punishes large mistakes more, used for regression
- **MAE (Mean Absolute Error)** — uses absolute error, treats mistakes more evenly
- **Cross-Entropy Loss** — measures confidence of correctness, used for classification

b. Error Minimization

SIMPLE:

Error minimization is the process of repeatedly adjusting a model to shrink its cost function.

EXPERT:

Error minimization is the iterative process by which a model's parameters are updated to reduce the value of the cost function, ideally converging toward its minimum.

c. Gradient Intuition

SIMPLE:

The gradient is the slope of the cost function, showing which way and how much to adjust weights.

EXPERT:

The gradient is the derivative of the cost function with respect to model parameters, providing both the direction and magnitude of the steepest ascent; parameters are updated in the opposite direction to minimize cost.

d. Optimization Workflows

SIMPLE:

The training cycle is: predict, measure error, compute gradient, adjust weights, repeat.

EXPERT:

An optimization workflow is the iterative gradient descent loop -- forward pass, loss computation, gradient calculation, and parameter update -- executed repeatedly until convergence.

TYPES:

- **Batch Gradient Descent** — uses the entire dataset per update; accurate but slow
- **Stochastic Gradient Descent (SGD)** — uses one sample per update; fast but noisy
- **Mini-batch Gradient Descent** — uses small groups of samples per update; the common default

e. Learning Through Iteration

SIMPLE:

Models learn gradually by repeating the predict-adjust cycle many times, not in one shot.

EXPERT:

Learning through iteration refers to the epoch-based convergence process in which repeated gradient updates progressively reduce loss, with training typically halted once improvement plateaus.

Module 6: ML Mathematics in Practice

a. Linear Regression Intuition

SIMPLE:

Linear regression finds the best straight line through data to predict a number.

EXPERT:

Linear regression models the relationship between input features and a continuous target as a weighted linear combination plus bias, with optimal weights found by minimizing MSE via gradient descent or the closed-form Normal Equation.

b. Similarity Calculations

SIMPLE:

Similarity calculations (like cosine similarity) measure how alike two data points are.

EXPERT:

Similarity calculations, most commonly cosine similarity, quantify the closeness of vector representations and underpin search ranking, semantic matching, and collaborative-filtering recommendation systems.

c. Distance Measurements

SIMPLE:

Distance measurements show how far apart two points are, the opposite of similarity.

EXPERT:

Distance measurements such as Euclidean and Manhattan distance quantify dissimilarity between vectors and are the core mechanism behind instance-based algorithms such as K-Nearest Neighbors and K-Means clustering.

TYPES:

- **Euclidean distance** — straight-line distance, most common
- **Manhattan distance** — grid-based distance, no diagonal shortcuts
- **Cosine distance** — 1 minus cosine similarity, measures angular difference

d. Feature Transformations

SIMPLE:

Feature transformation means changing feature values into a suitable scale before training.

EXPERT:

Feature transformation is the process of converting feature values into a suitable scale or representation before training, commonly via normalization to a 0-1 range, preventing large-scale features from dominating distance- or gradient-based algorithms.

e. Mathematical Reasoning in ML

SIMPLE:

This means reading a problem and figuring out which math tool fits, not just running code blindly.

EXPERT:

Mathematical reasoning in ML is the diagnostic skill of mapping a real-world problem to the correct mathematical formulation -- vectors and similarity for recommendation, matrices and gradients for regression -- prior to implementation.